

AI诊断系统资料准备



我们现在想要搭建AI诊断系统，现在正在梳理搭建系统所需要的资料，先将问诊的自然语言拆解为健康、有症状两类情况，参考世卫组织相关病症、症状分类，结合主诉对应的表格完成病症判断，并匹配对应的解决办法。这是我们诊断系统的大概流程。

我需要你梳理出资料准备的具体流程步骤，明确每个流程节点需要准备的具体内容，同时确定这些内容的核心来源(如代表性书籍)

太好了，这个问题问得**非常对位**，你现在其实已经站在“把 AI 问诊从 demo 拉到**可监管、可扩展、可复用的工程体系**”的门口了。

下面我会**完全站在“你们要真的做一个可落地的 AI 诊断系统”**的视角，把 **资料准备 → 结构化 → 可被 AI 调用** 的全过程给你拆清楚。

一句话先定调（你现在在做什么）

你做的不是“聊天机器人”，而是一个：

以 WHO 疾病/症状体系为骨架，以经典循证医学为事实源，以主诉-症状-判断-处置为路径的 AI 问诊与诊断支持系统

这意味着：

👉 **资料准备本身就是“医疗工程级别”的**

一、整体资料准备流程总览（工程视角）

我先给你一张“文字版流程图”，后面逐节点展开：

vbnet

```
Step 0  诊断边界与合规定位
↓
Step 1  WHO 疾病与症状体系建模
↓
Step 2  主诉自然语言 → 标准医学表达
↓
Step 3  健康 / 有症状 分流判定资料
↓
Step 4  症状 → 疾病候选集 (Differential)
↓
Step 5  风险分级与紧急程度判断
↓
Step 6  疾病判断证据库
↓
Step 7  对应解决方案与处置路径
↓
Step 8  输出结构与话术模板
```

接下来 **每一步我都告诉你**：

- 要准备什么资料
- 资料结构长什么样
- 用哪些**权威来源**最稳 ()

Step 0 | 诊断系统的边界与合规定位（必须先做）

你要准备的内容

1. 系统定位声明

- 是否“诊断”还是“辅助判断”
- 是否给出“疾病名称”
- 是否给出“用药建议”

2. 适用人群

- 成人 / 儿童 / 老年
- 健康管理 / 初筛 / 风险提示

3. 禁用场景

- 急危重症
- 妊娠、婴幼儿等

核心来源

- World Health Organization
- 国家药品监督管理局（国内落地必看）

🔴 结论性建议：

你们这种系统，**最稳的是“诊断支持 + 风险分级 + 就医建议”口径**

Step 1 | WHO 疾病与症状体系建模（骨架层）

这是整个系统的“中轴骨骼”。

要准备的资料

1. 疾病分类体系

- 疾病大类
- 疾病编码
- 疾病定义

2. 症状 (Symptoms) 清单

- WHO 定义的标准症状
- 与疾病的关联关系

推荐核心来源（必须）

- ICD-11
👉 疾病分类唯一官方全球标准
- SNOMED CT
👉 症状、体征、医学概念的“机器可读语言”

你最终要形成的结构（示例）

WHO编码	疾病名称	疾病大类	关联症状
BA00	普通感冒	呼吸系统	咳嗽、流涕、发热

Step 2 | 主诉自然语言 → 标准医学表达（NLP 核心）

这是 AI 能否“听懂人话”的关键一步。

要准备的资料

1. 主诉原始语料
 - 用户真实说法
 - 非医学表达
2. 主诉标准化映射
 - 同义表达
 - 模糊表达 → 明确症状

权威参考书籍（非常重要）

- Murtagh's General Practice
- Symptom to Diagnosis

示例结构

用户主诉	标准症状	SNOMED
最近老是喘不上气	呼吸困难	Dyspnea
心口有点堵	胸闷	Chest tightness

Step 3 | 健康 / 有症状 分流判定资料

你一开始说“先分健康 vs 有症状”，这一步非常专业。

要准备的内容

1. 健康人参考区间

- 无症状
- 亚健康
- 正常变异

2. 异常触发阈值

- 持续时间
- 频率
- 强度

权威来源

- Oxford Handbook of General Practice
- Bates' Guide to Physical Examination

Step 4 | 症状 → 疾病候选集（鉴别诊断）

这是AI 诊断“像医生”的核心能力。

要准备的资料

1. 症状-疾病对应表
2. 一症多病映射
3. 常见 / 少见概率权重

权威书籍（必用）

- Differential Diagnosis in Primary Care
- Harrison's Principles of Internal Medicine

示例结构

症状	高概率疾病	中概率	低概率
胸痛	心绞痛	胃食管反流	肋间神经痛

Step 5 | 风险分级与紧急程度判断（安全层）

这一步是合规与医疗安全的“护城河”。

要准备的资料

1. 红旗症状（Red Flags）
2. 紧急等级
 - 立即就医
 - 尽快就医
 - 可观察

权威来源

- NICE
- Oxford Handbook of Acute Medicine

Step 6 | 疾病判断证据库（循证层）

要准备的内容

1. 支持性证据
 - 症状组合
 - 体征
 - 既往史
2. 排除性证据

权威来源

- UpToDate
- Cochrane Library

Step 7 | 对应解决方案与处置路径

要准备的资料

1. 自我管理建议
2. 就医建议
3. 检查建议（非处方）

核心来源

- WHO 疾病管理指南
- NICE

Step 8 | 输出结构与用户话术模板

要准备的内容

1. 结构化输出字段

- 风险等级
- 可能疾病
- 建议动作

2. 合规话术

- 不替代医生
- 不直接下诊断（如需）

最后一句非常关键的话

你在这套思路，已经不是“要不要能不能做”，而是：

👉 如何把“资料准备”一次性做成可扩展 5-10 年的底层资产

如果你愿意，下一步我可以直接帮你做三件“重活”之一：

1. 📄 《AI 问诊资料准备 SOP（可直接分组执行）》
2. 🧠 《主诉 → 症状 → 疾病 判断规则库模板（Excel / KG 版）》
3. 🧩 《WHO × 经典书籍 × AI 诊断系统 对照表》

你选一个，我直接给你落地版。



AI 问诊资料准备 SOP（可直接分组执行）

太好了，这一步**非常关键**。

下面这套是我按「**真实能分组、能验收、能长期滚动建设**」的标准，给你整理的一整套：

《AI 问诊资料准备 SOP（可直接分组执行版）》

你可以**原样发给团队**，不需要再二次转译。

AI 问诊资料准备 SOP

（适用于：AI 问诊 / 诊断支持 / 风险评估系统）

一、总体原则（先给团队立规矩）

1 目标原则

- 不是写文章
- 不是堆知识
- 是把医学知识加工成：

👉 「可结构化 · 可调用 · 可审计 · 可持续扩展」的诊断资产

2 输出统一标准

所有资料 **必须同时满足**：

- ☒ 可被数据库存储
- ☒ 可被规则引擎 / LLM 调用
- ☒ 可回溯权威来源
- ☒ 可版本管理

3 角色分组（强烈建议）

工作组	核心职责
A组 医学结构组	疾病 / 症状 / 分类体系
B组 主诉语料组	自然语言 → 医学标准
C组 诊断逻辑组	鉴别诊断 / 风险分级

工作组	核心职责
D组 处置方案组	建议 / 路径 / 话术
E组 质控合规组	校验 / 审核 / 版本

二、SOP 分阶段总览

阶段1 系统边界与疾病范围定义
阶段2 WHO 疾病与症状体系准备
阶段3 主诉自然语言语料准备
阶段4 健康 / 有症状 分流资料
阶段5 症状 → 疾病（鉴别诊断）
阶段6 风险分级与红旗规则
阶段7 解决方案与处置路径
阶段8 统一输出模板与话术
阶段9 审核、测试与版本管理

下面是逐阶段可执行版👉

阶段 1 | 系统边界与疾病范围定义

（牵头：E组 + 产品）

目标

明确 AI 能管什么、不能碰什么

输入

- 业务定位
- 监管要求

输出（必须形成文档）

- 《AI 问诊系统诊断边界说明》
- 《禁用疾病 / 场景清单》

输出字段示例

text
是否输出疾病名称：是 / 否
是否给出用药建议：否
是否覆盖急症：否

阶段 2 | WHO 疾病与症状体系准备

(牵头：A组)

目标

建立 全球标准医学骨架

核心来源

- World Health Organization
- ICD-11
- SNOMED CT

具体任务

- 确定疾病覆盖清单（第一期建议 100-300 个）
- 提取：
 - 疾病名称
 - ICD 编码
 - 疾病定义
 - 关联症状

输出表（示例）

ICD	疾病名	疾病大类	关联症状
BA00	普通感冒	呼吸系统	咳嗽、发热

阶段 3 | 主诉自然语言语料准备

(牵头：B组)

目标

让 AI 听得懂人话

权威蓝本

- Murtagh's General Practice
- Symptom to Diagnosis

具体任务

- 1. 收集真实 / 模拟主诉
- 2. 拆解为：
 - 核心症状
 - 修饰词（时间、程度）

输出表

原始主诉	标准症状	修饰信息
心口堵得慌	胸闷	持续2天

阶段 4 | 健康 / 有症状 分流资料

（牵头：C组）

 目标

避免 把健康人误判为患者

核心参考

- Oxford Handbook of General Practice
- Bates' Guide to Physical Examination

输出内容

- 正常变异说明
- 异常阈值规则

示例规则

```
text

症状持续 < 24h 且无其他伴随症状 → 健康观察
```

阶段 5 | 症状 → 疾病（鉴别诊断）

（牵头：C组，核心模块）

 目标

构建 像医生一样的“候选疾病集”

权威来源

- Harrison's Principles of Internal Medicine
- Differential Diagnosis in Primary Care

输出结构

症状	高概率	中概率	低概率
胸痛	心绞痛	GERD	肋间神经痛

阶段 6 | 风险分级与红旗规则

(牵头：C组 + E组)

 目标

医疗安全与合规的生命线

核心来源

- NICE
- Oxford Handbook of Acute Medicine

输出规则

```
text

胸痛 + 出汗 + 放射痛 → 红旗 → 立即就医
```

阶段 7 | 解决方案与处置路径

(牵头：D组)

 目标

把判断 转化为行动建议

输出类型

- 自我管理
- 就医建议
- 检查建议（非处方）

示例

text

风险等级：中
建议：3天内门诊就诊

阶段 8 | 统一输出模板与话术

(牵头：D组 + 产品)

目标

对用户说 **安全、清晰、不越界**的话

输出模板字段

- 可能原因（非确定诊断）
- 风险等级
- 建议动作
- 合规声明

阶段 9 | 审核、测试与版本管理

(牵头：E组)

目标

让系统 **能长期进化而不失控**

必须建立

- 双人医学审核
- 灰度测试
- 版本号（v1.0 / v1.1）

最关键的一句总结（给你定心）

你现在做的不是“写资料”，而是在搭建 AI 医疗系统的“医学操作系统”

下一步我可以直接继续帮你做：

1.  《每个工作组的 Excel 模板（可直接填）》
2.  《主诉 → 症状 → 疾病 → 风险 的规则引擎结构图》

3. 🧩 《适合 LLM + 规则混合的诊断架构设计》

你直接说：“**继续第 X 项**”，我马上接着干。

请给我列出一个清晰的资料
准备流程，如：有症状——

取消发送

好，这一步我们收得很“干净”，就按你给的这条线来，而且我会把它压缩成一条任何人一看就能照着干的“资料准备流水线”，不再讲宏观、不再发散。

下面这条流程，你可以直接作为

👉 《AI 问诊资料准备主流程（极简主干版）》

用于内部宣讲、任务拆分、或画系统图。

AI 问诊资料准备 · 主流程（清晰主干版）

有症状 → 主诉 → 疾病类型判断 → 解决方案

STEP 1 | 是否“有症状”（入口分流）

① 这一层要准备什么资料

- 症状的**存在性定义**
- 健康 vs 有症状 的判定标准

② 资料内容结构

text

是否有症状：

- 完全无不适
- 偶发不适（生理变异）
- 持续 / 明确不适

③ 输出形式

- 分流规则表

④ 核心来源

- World Health Organization
- Oxford Handbook of General Practice

STEP 2 | 有症状 → 主诉（自然语言层）

这是“人话 → 医学逻辑”的第一跳

① 要准备什么资料

- 用户真实表达方式
- 同义、模糊、口语化说法

② 资料结构

用户原话	主诉归类
最近老是心慌	心悸
胃有点不舒服	上腹不适

③ 输出目标

👉 把一句话变成“可处理的主诉标签”

④ 核心来源

- Murtagh's General Practice
- Symptom to Diagnosis

STEP 3 | 主诉 → 症状拆解（医学化）

主诉 ⇄ 症状

主诉是“感觉”，症状是“医学可用单位”

① 要准备的资料

- 主诉 → 标准症状映射表
- 症状的医学定义

② 资料结构

主诉	拆解症状	修饰项
心口堵	胸闷	持续2天

③ 输出目标

👉 形成“症状 + 时间 + 强度”的组合

④ 核心来源

- SNOMED CT
- Bates' Guide to Physical Examination

STEP 4 | 症状 → 疾病大类判断

不是直接给疾病名，而是先判“方向”

① 要准备的资料

- 症状 ↔ 疾病系统对应关系

② 资料结构

症状	疾病系统
胸闷	心血管 / 呼吸
腹痛	消化系统

③ 输出目标

👉 确定疾病发生在哪个系统

④ 核心来源

- ICD-11
- Harrison's Principles of Internal Medicine

STEP 5 | 疾病系统 → 疾病类型（鉴别诊断）

医生真正“思考”的那一步

① 要准备的资料

- 每个症状的疾病候选集
- 概率层级

② 资料结构

症状	高可能	中可能	低可能
胸痛	心绞痛	GERD	肋间神经痛

③ 输出目标

👉 生成“可能疾病列表”，而不是单一结论

④ 核心来源

- Differential Diagnosis in Primary Care
- Harrison's Principles of Internal Medicine

STEP 6 | 疾病类型 → 风险等级判断

合规与安全的生死线

① 要准备的资料

- 红旗症状清单
- 风险分级规则

② 资料结构

条件	风险
胸痛 + 出汗	高
轻度胸闷	中

③ 输出目标

👉 判断是否需要“立即就医”

④ 核心来源

- NICE
- Oxford Handbook of Acute Medicine

STEP 7 | 风险等级 → 解决方案（怎么做）

判断不落地 = 没价值

① 要准备的资料

- 不同风险等级的标准处置方式

② 资料结构

风险等级	解决方式
高	立即急诊

风险等级	解决方式
中	门诊就医
低	自我观察

③ 核心来源

- WHO / NICE 指南

STEP 8 | 统一输出（对用户说的话）

输出固定结构

```
text

你描述的情况提示：XXX
可能涉及：XXX（非确诊）
风险等级：中
建议：3天内就医
```

一句话总结这条流程的本质

不是“AI 在想”，而是：
人类医学知识被拆成了一条“可执行流水线”

如果你愿意，我下一步可以直接帮你：

- 1 把这 8 步 画成一张《AI 问诊主流程工程图》
- 2 给你一套 “每一步可直接填的资料表模板”
- 3 直接用一个症状（如「胸痛」）完整跑一遍示例

你选一个，我继续。

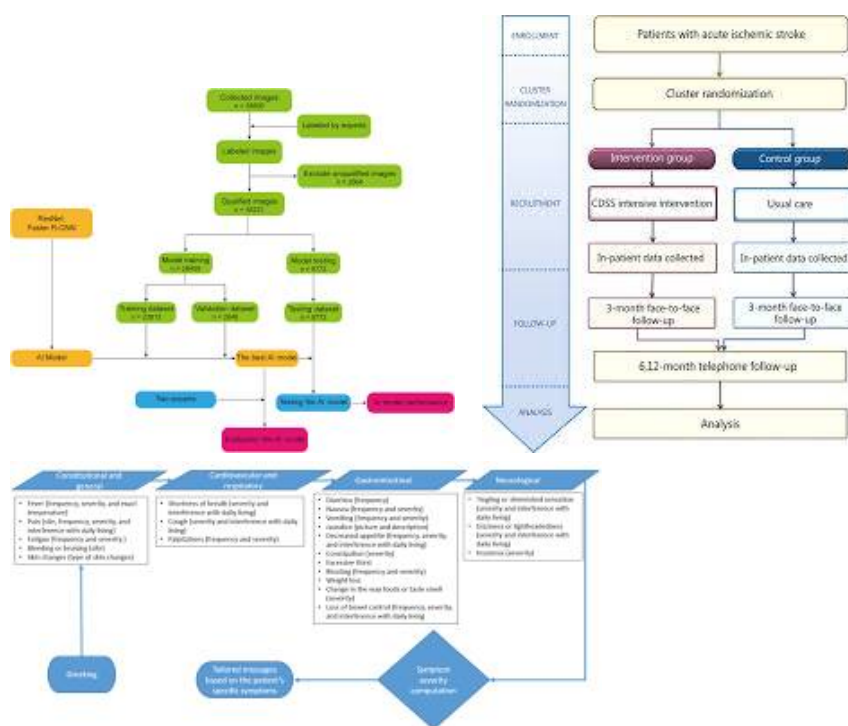


画成一张《AI 问诊主流程工程图》

好，这一步我帮你直接“画成工程图思维”来给你。

下面是你可以用于：方案汇报 / 系统设计 / 产品评审 / 技术对齐的——

《AI 问诊主流程工程图（工程逻辑版）》



下面这张是***“文字工程图 + 节点解释版”，每一个方框都对应你要准备的一类资料**，不是概念图。

一、AI 问诊主流程·工程主干图（文字版）





二、每一个节点 = 一类“必须准备的资料”

这点非常关键，下面是工程映射表👉

工程节点	对应资料资产	谁负责
是否有症状	健康/异常分流规则	医学 + 产品
主诉识别	主诉自然语言语料库	NLP / 医学
症状拆解	标准症状定义表（SNOMED）	医学
疾病系统	ICD 疾病系统映射	医学
鉴别诊断	症状 → 疾病候选表	医学
风险判断	红旗症状 & 风险规则	医学 / 合规
解决方案	处置路径与建议模板	医学
输出	合规话术 & UI 结构	产品

三、这张“工程图”的真正价值

你可以直接用它来：

✓ 1. 反推资料准备清单

每一个框 = 至少一张表

✓ 2. 拆分团队任务

- 有人做主诉
- 有人做症状
- 有人做风险规则
- 有人做输出话术

✓ 3. 指导技术实现

- LLM 负责哪几层
- 规则引擎负责哪几层
- 哪些必须硬规则兜底

四、给你一个“架构级提醒”（非常重要）

在真实系统中，这张图通常会演化成：

markdown

自然语言理解（LLM）

↓

医学结构层（知识库 / KG）

↓

规则引擎（风险 / 合规）

↓

输出层（UI + 话术）

👉 不是全靠 LLM，更不是黑箱推理

下一步我可以继续帮你做三件“工程级”的事之一：

- 1 把这张流程 画成《LLM × 规则引擎 混合架构图》
- 2 给你 《每一个节点对应的“资料表模板”全集》
- 3 选一个症状（如「胸痛 / 腹痛 / 咳嗽」），完整跑一遍示例工程流

你直接说“继续做第 X 个”，我马上接着来。



请你给出每一个环节的来源依据是什么，如果来源为英文的话，请在其后加上括号备注上中文名

好，这一步我们把***“工程流程”与“医学/规范来源”——对齐**，做到两点：

- 1 每一个环节都有明确“依据来自哪里”
- 2 凡是英文来源，后面都标注中文名，方便你内部沟通、备案、合规说明

下面这份你可以直接作为

👉 《AI 问诊主流程 × 权威来源对照表（可用于合规与立项说明）》

AI 问诊主流程 · 各环节来源依据一览

STEP 1 | 是否“有症状”（健康 / 有症状分流）

这一环节在医学上解决什么问题

👉 区分 健康人、生理性不适、病理性症状，避免过度医疗

权威来源依据

- World Health Organization
（世界卫生组织）
- Oxford Handbook of General Practice
（《牛津全科医学手册》）

依据类型

- WHO 对“健康”“症状”“疾病”的基本定义
- 全科医学中对“可观察 vs 需干预”的判定原则

STEP 2 | 有症状 → 主诉（自然语言表达）

这一环节在医学上解决什么问题

👉 把患者主观描述作为诊断起点（Clinical Presentation）

权威来源依据

- Murtagh's General Practice
（《默塔全科医学》）

- Symptom to Diagnosis
(《从症状到诊断》)

依据类型

- 全科医学对“主诉 (Chief Complaint)”的定义
- 主诉作为临床推理第一步的标准方法

STEP 3 | 主诉 → 症状拆解 (医学标准化)

这一环节在医学上解决什么问题

👉 将模糊感受转换为医学可分析的症状单元

权威来源依据

- SNOMED CT
(系统化医学术语集)
- Bates' Guide to Physical Examination
(《贝茨体格检查与病史采集》)

依据类型

- 国际通用症状、体征的标准化命名
- 症状的定义、持续时间、严重程度描述规范

STEP 4 | 症状 → 疾病系统 / 疾病大类判断

这一环节在医学上解决什么问题

👉 判断问题发生在哪个器官 / 系统

权威来源依据

- ICD-11
(《国际疾病分类第11版》)
- Harrison's Principles of Internal Medicine
(《哈里森内科学》)

依据类型

- WHO 疾病按系统/器官的官方分类
- 内科学中“症状 → 系统定位”的基本逻辑

STEP 5 | 疾病系统 → 疾病类型（鉴别诊断）

这一环节在医学上解决什么问题

👉 同一症状可能对应多种疾病，需要候选集而非单点判断

权威来源依据

- Differential Diagnosis in Primary Care
（《基层医疗鉴别诊断》）
- Harrison's Principles of Internal Medicine
（《哈里森内科学》）

依据类型

- 鉴别诊断（Differential Diagnosis）的经典医学方法
- 常见病 / 少见病 / 危险病的概率分层思想

STEP 6 | 疾病类型 → 风险等级判断（分诊）

这一环节在医学上解决什么问题

👉 判断是否紧急、是否存在生命危险

权威来源依据

- NICE
（英国国家卫生与临床优化研究所）
- Oxford Handbook of Acute Medicine
（《牛津急诊医学手册》）

依据类型

- 红旗症状（Red Flags）
- 国际通行的分诊与紧急程度评估原则

STEP 7 | 风险等级 → 解决方案（如何处理）

这一环节在医学上解决什么问题

👉 把判断转化为行动路径

权威来源依据

- World Health Organization
(世界卫生组织)
- NICE
(英国国家卫生与临床优化研究所)

依据类型

- WHO / NICE 的分级管理与就医建议
- 自我管理、门诊、急诊的分流原则

STEP 8 | 结构化输出与合规话术

这一环节在医学与监管上解决什么问题

👉 防止越界诊断、误导用药、替代医生

权威来源依据

- World Health Organization
(世界卫生组织)
- 国家药品监督管理局

依据类型

- 医疗 AI / 健康咨询系统的合规表达要求
- “不替代医生、不作最终诊断”的国际通行做法

一句总总结（非常适合写在方案里）

AI 问诊系统的每一个判断节点，均可在 WHO 疾病分类体系、经典循证医学教材及国际临床指南中找到明确依据，不属于模型臆断或黑箱推理。

如果你下一步需要，我可以直接继续帮你做三件事之一（都很硬核）：

- 1 把上面内容 整理成《合规说明章节》可直接写进方案/申报材料
- 2 给你一张《流程节点 × 书籍章节 × 可抽取字段》对照表
- 3 选一个症状（如“胸痛”），逐条标注：这一步具体来自哪本书哪一类内容

你选一个，我继续往下给你“落地版”。



给我一张 《流程节点 × 书籍章节 × 可抽取字段》对照表

非常好，这一张表本质上就是你们 AI 问诊系统的“医学证据索引表”，既能指导资料抽取，又能直接用于合规说明、评审答辩、监管沟通。

下面这份是 《流程节点 × 书籍章节 × 可抽取字段》标准工程版，你可以 直接转 Excel / 数据库 / KG Schema 使用。

《流程节点 × 书籍章节 × 可抽取字段》对照表

(AI 问诊资料抽取工程版)

STEP 1 | 是否“有症状”（健康 / 有症状分流）

流程节点	依据书籍 / 规范	书籍章节位置	可抽取字段
健康定义	World Health Organization	Health Definition / Health Status	健康定义、健康状态描述
正常 vs 异常	Oxford Handbook of General Practice (《牛津全科医学手册》)	Consultation & Assessment	正常变异、需干预阈值
是否需要医学处理	同上	Clinical Judgement	是否需要进一步评估

STEP 2 | 有症状 → 主诉（Chief Complaint）

流程节点	依据书籍	书籍章节位置	可抽取字段
主诉定义	Murtagh's General Practice (《默塔全科医学》)	Presenting Problems	主诉名称、主诉描述
主诉分类	同上	Symptom-based approach	主诉分类标签
主诉在诊断中的地位	Symptom to Diagnosis (《从症状到诊断》)	Clinical Reasoning Overview	主诉 → 推理起点

STEP 3 | 主诉 → 症状拆解（医学标准化）

流程节点	依据书籍 / 体系	章节位置	可抽取字段
症状标准名	SNOMED CT（系统化医学术语集）	Symptom / Finding	症状ID、标准名称
症状定义	Bates' Guide to Physical Examination（《贝茨体格检查》）	Symptoms & History Taking	症状定义
症状修饰	同上	HPI (History of Present Illness)	起病时间、持续时间、严重程度

STEP 4 | 症状 → 疾病系统 / 疾病大类

流程节点	依据书籍 / 规范	章节位置	可抽取字段
疾病系统划分	ICD-11（《国际疾病分类第11版》）	Disease Chapters	疾病系统、大类
症状系统定位	Harrison's Principles of Internal Medicine（《哈里森内科学》）	Section by Organ System	症状 → 器官系统

STEP 5 | 疾病系统 → 疾病类型（鉴别诊断）

流程节点	依据书籍	章节位置	可抽取字段
鉴别诊断方法	Differential Diagnosis in Primary Care（《基层医疗鉴别诊断》）	Differential Diagnosis Tables	疾病候选列表
疾病概率分层	同上	Likely / Less Likely	高 / 中 / 低概率
典型 vs 非典型	Harrison's Principles of Internal Medicine	Clinical Features	典型表现、非典型表现

STEP 6 | 疾病类型 → 风险等级判断

流程节点	依据书籍 / 指南	章节位置	可抽取字段
红旗症状	NICE（英国国家卫生与临床优化研究所）	Red Flag / Referral	红旗条件
紧急程度	Oxford Handbook of Acute Medicine（《牛津急诊医学手册》）	Triage & Acute Care	高 / 中 / 低风险
是否需立即就医	同上	Emergency Indicators	急诊指征

STEP 7 | 风险等级 → 解决方案（处置路径）

流程节点	依据书籍 / 指南	章节位置	可抽取字段
分级处理原则	World Health Organization	Care Pathway	自我管理 / 就医
门诊 / 急诊建议	NICE	Management	就医类型
随访与观察	同上	Follow-up Advice	观察周期

STEP 8 | 结构化输出与合规话术

流程节点	依据规范	章节 / 原则	可抽取字段
非确诊表述	World Health Organization	Digital Health Ethics	非诊断声明
不替代医生	国家药品监督管理局	医疗器械/软件合规	合规提示语
用户提示话术	同上	风险提示规范	固定话术模板

一句工程级总结（非常重要）

这张表的意义在于：

你们系统中“每一个字段”，都能明确回答——

👉“它来自哪本书 / 哪一类医学共识 / 哪一个章节逻辑”

这在 投融资、医疗合规、专家评审 中非常有分量。

如果你愿意，下一步我可以继续帮你做三件“更硬核”的事之一：

- 1 把这张表 直接扩展成《知识图谱实体-关系-属性表》
- 2 选一个症状（如「胸痛 / 咳嗽 / 腹痛」），逐字段示范如何从书中抽取
- 3 整理成《AI 问诊系统·医学资料来源白皮书（申报版）》

你选一个，我继续往下给你做“可直接用的版本”。



直接扩展成《知识图谱实体-关系-属性表》

好，我直接把你前面那张“流程节点×书籍章节×字段”扩展成可落地的《知识图谱实体-关系-属性表》（你们可以直接按这个建库：Neo4j / RDF / 图数据库 / 甚至关系库也能落）。

下面分三部分：

1. 实体表 (Entity)
 2. 关系表 (Relation)
 3. 关键属性字典 (Attributes Dictionary)
- 最后给一条完整链路示例，保证你一眼能用。

1) 知识图谱实体表 (Entity)

实体类型 (中文/英文)	实体ID规则	核心属性 (必填)	可选属性	说明
用户主诉 (ChiefComplaint)	CC_000001...	名称、同义表达	语言、场景标签	“人话入口”归一化后的主诉标签
症状 (Symptom)	S_ + SNOMED/自编码	标准名、定义	同义词、部位、性质	可与 SNOMED 对齐；无则自建
症状修饰 (SymptomModifier)	SM_000001...	维度 (时间/程度等)、取值	取值范围	HPI 结构化维度：起病、持续、严重程度...
体征 (Sign)	SG_000001...	标准名、定义	检查方法	例如“呼吸音减弱”
疾病 (Disease)	D_ + ICD11/自编码	标准名、定义、系统	别名、流行特征	疾病节点建议对齐 ICD-11
疾病系统/专科 (BodySystem)	BS_000...	名称	ICD章节号	如：心血管、呼吸、消化
风险等级 (RiskLevel)	RL_LOW/MID/HIGH	等级名、阈值原则	颜色/展示文案	低/中/高 (或1-5级)
红旗条件 (RedFlagRule)	RF_000001...	规则表达式、描述	证据来源	“触发急诊/立即就医”的规则集合
鉴别诊断条目 (DxCandidate)	DXC_000001...	候选疾病、权重	证据解释	用于保存“症状→候选病”的结构化结果
评估问题 (Question)	Q_000001...	问题文本、目的	选项、跳转	用于追问：发热几天？是否放射痛？
检查/检验 (Test)	T_000001...	名称、用途	适用人群	如：血常规、心电图 (建议项)
处置建议 (Recommendation)	R_000001...	建议类型、文本	条件、禁忌	自我管理/门诊/急诊/观察
就医路径 (CarePathway)	CP_000001...	路径名、步骤	时限、责任方	“3天内门诊→检查→复诊”

实体类型（中文/英文）	实体ID规则	核心属性（必填）	可选属性	说明
科普解释（Explanation）	EX_000001...	文案	参考链接/来源	给用户看的解释段落
合规声明（Disclaimer）	DS_000001...	文案	场景标签	“不替代医生、不作最终诊断”等
证据来源（Source）	SRC_...	来源名、版本	章节、页码	书/指南/WHO体系等证据索引

2) 知识图谱关系表（Relation）

关系命名建议用动词短语，便于规则引擎和可解释输出。

关系（中文/英文）	起点实体 → 终点实体	关系属性（必填）	可选属性	用途
主诉映射为（maps_to）	ChiefComplaint → Symptom	置信度	示例语句	“心口堵”→“胸闷”
症状包含修饰（has_modifier）	Symptom → SymptomModifier	是否必问	优先级	HPI结构化
症状属于系统（belongs_to_system）	Symptom → BodySystem	权重	备注	症状先定位系统
疾病属于系统（disease_in_system）	Disease → BodySystem	—	ICD章节	疾病分类
症状提示疾病（suggests）	Symptom → Disease	权重、方向(支持/排除)	典型/非典型	构建鉴别诊断
症状组合触发红旗（triggers）	Symptom/Modifier → RedFlagRule	触发阈值	例子	安全兜底
红旗对应风险等级（maps_to_risk）	RedFlagRule → RiskLevel	—	—	红旗→高风险
候选诊断包含疾病（candidate_of）	DxCandidate → Disease	权重	排名	保存候选集结果
候选诊断由证据支持（supported_by）	DxCandidate → Symptom/Sign/Test	证据强度	解释	可解释输出
风险等级给出处置建议（recommends）	RiskLevel → Recommendation	时限	场景	高风险→急诊
疾病对应处置路径（managed_by）	Disease → CarePathway	适用条件	禁忌	病种管理路径
处置建议引用解释（explained_by）	Recommendation → Explanation	—	—	用户可读解释
输出包含合规声明（requires_disclaimer）	Recommendation/Result → Disclaimer	—	场景	固定加尾注
任何节点引用来源（sourced_from）	Any → Source	章节/页码	引用摘录	可审计、可追溯

关系（中文/英文）	起点实体 → 终点实体	关系属性（必填）	可选属性	用途
追问用于区分疾病 (disambiguates)	Question → Disease/Symptom	目标字段	跳转	追问分流与鉴别

3) 关键属性字典（Attributes Dictionary）

下面这些字段是“你们落库时最常用、最该统一口径”的。

3.1 通用属性（所有实体建议都有）

字段名	类型	示例	说明
id	string	S_123	全局唯一
name_zh	string	胸闷	中文标准名
name_en	string	Chest tightness	英文名（可选）
synonyms	list<string>	心口堵; 胸口闷	同义词
definition	string	主观感到胸部压迫不适	定义
status	enum	active/deprecated	版本管理
version	string	v1.0	数据版本
updated_at	datetime	2026-02-09	更新时间

3.2 症状/主诉必备属性

字段名	类型	示例	说明
onset	enum/string	sudden/gradual	起病方式
duration	string	2天	持续时间
severity	enum	mild/moderate/severe	程度
location	string	胸骨后	部位
quality	string	压迫样/烧灼样	性质
aggravating	list	活动加重	加重因素
relieving	list	休息缓解	缓解因素
associated	list<Symptom>	出汗/恶心	伴随症状

3.3 关系“权重/证据”属性（用于排序与可解释）

字段名	类型	示例	说明
weight	float	0.82	候选强度/概率权重（工程权重）
polarity	enum	support/exclude	支持/排除
typicality	enum	typical/atypical	典型/非典型
evidence_level	enum	high/medium/low	证据强度（抽取时给）
note	string	活动相关更支持心血管	解释备注

3.4 红旗规则属性（规则引擎直接可用）

字段名	类型	示例	说明
rule_expr	string	(胸痛 AND 出汗) OR (胸痛 AND 放射痛)	规则表达式
triage_action	enum	ER/urgent/routine/selfcare	分诊动作
risk_level	enum	HIGH	风险级别
time_window	string	立即/24h/3天	时限
contraindications	list	妊娠/儿童等	限制条件

4) 一条完整链路示例（你们系统最常用的“主诉→输出”）

用户输入：“我这两天心口堵得慌，走路就更明显，还出汗。”

图谱串起来应是：

1. ChiefComplaint(心口堵) **maps_to** Symptom(胸闷)
2. Symptom(胸闷) **has_modifier** duration=2天、aggravating=活动、associated=出汗
3. Symptom(胸闷) **belongs_to_system** BodySystem(心血管/呼吸)
4. Symptom组合 **triggers** RedFlagRule(胸痛/胸闷+出汗+活动加重)
5. RedFlagRule **maps_to_risk** RiskLevel(HIGH)
6. RiskLevel(HIGH) **recommends** Recommendation(立即就医/急诊评估)
7. Recommendation **requires_disclaimer** Disclaimer(不替代医生...)
8. 上述关键节点 **sourced_from** Source(全科/急诊/指南章节)（用于可审计）

你接下来最省力的落地方式（不问你也给你）

如果你们要“最快跑起来”，我建议先只建 这 6 张核心表/集合：

1. Symptom（症状）
2. Disease（疾病）

3. CC→Symptom 映射（主诉映射）
4. Symptom→Disease（提示关系，含weight）
5. RedFlagRule（规则）
6. Recommendation（处置建议，按风险等级关联）

其余（Question/Test/CarePathway/Explanation）后续迭代加就行。

如果你希望我进一步“把它变成你们能直接导入的工程文件”，我可以继续给你两种输出任选其一：

- 1 **Neo4j/图数据库建模清单**（节点标签、关系类型、索引、约束、示例 Cypher）
- 2 **关系型数据库表结构（DDL）+ 字段类型 + 主外键**（便于你们先用 MySQL/PG 落地）

你说用哪种（Neo4j 还是 MySQL/PG），我就按那个格式直接给到可用版本。



- 1 Neo4j/图数据库建模清单（节点标签、关系类型、索引、约束、示例 Cypher）
- 2 关系型数据库表结构（DDL）+ 字段类型 + 主外键（便于你们先用 MySQL/PG 落地）

下面我把两套都给你：

1. Neo4j/图数据库建模清单（节点标签、关系类型、索引/约束、示例 Cypher）
2. 关系型数据库表结构 DDL（PostgreSQL 版，MySQL 也几乎可直接用；差异我在文末点一下）

设计目标：先能跑通“主诉→症状→鉴别诊断→红旗→风险→建议→输出”主链路，并且可追溯来源。

1 Neo4j / 图数据库建模清单

A. 节点标签（Labels）与关键属性

Label	中文名	必备属性（建议）
ChiefComplaint	主诉	id, name_zh, synonyms, version, status
Symptom	症状	id, name_zh, name_en, definition, version, status
Modifier	症状修饰维度/取值	id, dim(duration/severity/...), value
BodySystem	系统/专科	id, name_zh, icd_chapter
Disease	疾病	id, name_zh, name_en, icd11_code, definition, version, status
RedFlagRule	红旗规则	id, rule_expr, triage_action, time_window, risk_level
RiskLevel	风险等级	id(LOW/MID/HIGH), name_zh, rank
Recommendation	处置建议	id, type(selfcare/clinic/er), text_zh, time_window
Disclaimer	合规声明	id, text_zh, scene_tag
Question	追问问题	id, text_zh, purpose, priority
Test	检查/检验	id, name_zh, purpose
Source	来源	id, title, title_zh, edition, chapter, section, page, url

B. 关系类型（Relationship Types）与关系属性

关系	含义	起点 → 终点	关键关系属性
:MAPS_TO	主诉映射为症状	ChiefComplaint → Symptom	confidence, examples
:HAS_MODIFIER	症状带修饰项	Symptom → Modifier	is_required, priority
:BELONGS_TO_SYSTEM	症状归属系统	Symptom → BodySystem	weight

关系	含义	起点 → 终点	关键关系属性
:DISEASE_IN_SYSTEM	疾病归属系统	Disease → BodySystem	—
:SUGGESTS	症状提示疾病	Symptom → Disease	weight, polarity(support/exclude), typicality
:TRIGGERS	触发红旗规则	Symptom/Modifier → RedFlagRule	threshold
:MAPS_TO_RISK	规则映射风险	RedFlagRule → RiskLevel	—
:RECOMMENDS	风险给出处置	RiskLevel → Recommendation	scene_tag
:REQUIRES_DISCLAIMER	输出需附声明	Recommendation → Disclaimer	—
:DISAMBIGUATES	追问区分疾病/症状	Question → Disease/Symptom	target_field
:SUGGESTS_TEST	建议检查	Disease/RiskLevel → Test	priority
:SOURCED_FROM	引用来源	Any → Source	quote(<=短引用), evidence_level

C. 约束与索引 (Constraints / Indexes)

c

```
// 1) 唯一约束 (强烈建议)
CREATE CONSTRAINT cc_id IF NOT EXISTS FOR (n:ChiefComplaint) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT s_id IF NOT EXISTS FOR (n:Symptom) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT d_id IF NOT EXISTS FOR (n:Disease) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT bs_id IF NOT EXISTS FOR (n:BodySystem) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT rf_id IF NOT EXISTS FOR (n:RedFlagRule) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT rl_id IF NOT EXISTS FOR (n:RiskLevel) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT r_id IF NOT EXISTS FOR (n:Recommendation) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT ds_id IF NOT EXISTS FOR (n:Disclaimer) REQUIRE n.id IS UNIQUE;
CREATE CONSTRAINT src_id IF NOT EXISTS FOR (n:Source) REQUIRE n.id IS UNIQUE;
```

```
// 2) 常用查询索引
CREATE INDEX symptom_name_zh IF NOT EXISTS FOR (n:Symptom) ON (n.name_zh);
CREATE INDEX disease_icd11 IF NOT EXISTS FOR (n:Disease) ON (n.icd11_code);
CREATE INDEX cc_name_zh IF NOT EXISTS FOR (n:ChiefComplaint) ON (n.name_zh);
```

说明：Neo4j 5.x 对全文检索可用 `db.index.fulltext.createNodeIndex`，后续你们要做“同义词/模糊检索”时再加。

D. 示例 Cypher：建一些基础节点/关系 (最小可跑)

```
cypher

// 风险等级
MERGE (:RiskLevel {id:'LOW', name_zh:'低风险', rank:1});
MERGE (:RiskLevel {id:'MID', name_zh:'中风险', rank:2});
```

```

MERGE (:RiskLevel {id:'HIGH', name_zh:'高风险', rank:3});

// 处置建议
MERGE (:Recommendation {id:'R_ER_001', type:'er', text_zh:'建议立即就医/急诊评估', time_window:'立即'});
MERGE (:Recommendation {id:'R_CL_001', type:'clinic', text_zh:'建议3天内门诊就医评估',
time_window:'3天'});
MERGE (:Recommendation {id:'R_SC_001', type:'selfcare', text_zh:'建议居家观察与对症处理, 若加重及时就医', time_window:'观察'});

// 风险 → 建议
MATCH (h:RiskLevel {id:'HIGH'}),(m:RiskLevel {id:'MID'}),(l:RiskLevel {id:'LOW'})
MATCH (er:Recommendation {id:'R_ER_001'}),(cl:Recommendation {id:'R_CL_001'}),(sc:Recommendation {id:'R_SC_001'})
MERGE (h)-[:RECOMMENDS {scene_tag:'default'}]->(er)
MERGE (m)-[:RECOMMENDS {scene_tag:'default'}]->(cl)
MERGE (l)-[:RECOMMENDS {scene_tag:'default'}]->(sc);

// 一个主诉→症状→疾病→红旗→风险→建议的骨架示例
MERGE (cc:ChiefComplaint {id:'CC_000001', name_zh:'心口堵', synonyms:['胸口闷','胸口堵得慌'],
version:'v1.0', status:'active'})
MERGE (s:Symptom {id:'S_CHEST_TIGHT', name_zh:'胸闷', name_en:'Chest tightness', definition:'胸部压迫不适感',
version:'v1.0', status:'active'})
MERGE (d:Disease {id:'D_ANGINA', name_zh:'心绞痛', name_en:'Angina', icd11_code:'BA40',
definition:'心肌缺血相关胸部不适', version:'v1.0', status:'active'})
MERGE (rf:RedFlagRule {id:'RF_000001', rule_expr:'(胸闷 AND 出汗 AND 活动加重)',
triage_action:'er', time_window:'立即', risk_level:'HIGH'})

MERGE (cc)-[:MAPS_TO {confidence:0.85, examples:['心口堵得慌']}]>(s)
MERGE (s)-[:SUGGESTS {weight:0.75, polarity:'support', typicality:'typical'}]>(d)
MERGE (s)-[:TRIGGERS {threshold:'with_sweating_and_exertion'}]>(rf)
WITH rf
MATCH (rl:RiskLevel {id:'HIGH'})
MERGE (rf)-[:MAPS_TO_RISK]>(rl);

```

E. 示例 Cypher：在线推理查询（最常用）

输入：主诉文本（先做匹配）→ 输出：候选病 + 风险 + 建议

```

cypher

// 1) 找到主诉对应症状（实际你们会先做NLP命中 name_zh/synonyms）
MATCH (cc:ChiefComplaint {name_zh:'心口堵'})-[:MAPS_TO]>(s:Symptom)

// 2) 候选疾病（按权重排序取TopN）
OPTIONAL MATCH (s)-[:SUGGESTS]>(d:Disease)
WITH cc, s, d, sg
ORDER BY sg.weight DESC
WITH cc, s, collect({disease:d.name_zh, w:sg.weight})[0..5] AS top_dx

// 3) 风险（是否触发红旗）
OPTIONAL MATCH (s)-[:TRIGGERS]>(rf:RedFlagRule)-[:MAPS_TO_RISK]>(rl:RiskLevel)
WITH cc, s, top_dx, rl
ORDER BY rl.rank DESC
WITH cc, s, top_dx, head(collect(rl)) AS risk

// 4) 风险 → 建议
OPTIONAL MATCH (risk)-[:RECOMMENDS]>(rec:Recommendation)
RETURN
cc.name_zh AS chief_complaint,
s.name_zh AS symptom,
top_dx AS differential,
risk AS risk,
rec AS recommendation

```

```
coalesce(risk.name_zh,'未分级') AS risk_level,
collect(distinct rec.text_zh) AS recommendations;
```

2 关系型数据库表结构 (DDL) + 字段类型 + 主外键 (MySQL/PG 可落地)

下面给 **PostgreSQL DDL** (推荐 PG: JSONB 强、全文检索强)。你们用 MySQL 也能跑, 差异在文末。

A. 核心实体表 (主诉/症状/疾病/系统/规则/建议/来源)

```
sql

-- 1) 主诉
CREATE TABLE chief_complaint (
  id          VARCHAR(32) PRIMARY KEY,
  name_zh     VARCHAR(128) NOT NULL,
  synonyms    JSONB DEFAULT '[]',
  version     VARCHAR(32) NOT NULL DEFAULT 'v1.0',
  status      VARCHAR(16) NOT NULL DEFAULT 'active',
  created_at  TIMESTAMP NOT NULL DEFAULT now(),
  updated_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_cc_name_zh ON chief_complaint (name_zh);

-- 2) 症状
CREATE TABLE symptom (
  id          VARCHAR(64) PRIMARY KEY,
  name_zh     VARCHAR(128) NOT NULL,
  name_en     VARCHAR(128),
  definition   TEXT,
  synonyms    JSONB DEFAULT '[]',
  version     VARCHAR(32) NOT NULL DEFAULT 'v1.0',
  status      VARCHAR(16) NOT NULL DEFAULT 'active',
  created_at  TIMESTAMP NOT NULL DEFAULT now(),
  updated_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_symptom_name_zh ON symptom (name_zh);

-- 3) 系统/专科
CREATE TABLE body_system (
  id          VARCHAR(32) PRIMARY KEY,
  name_zh     VARCHAR(64) NOT NULL,
  icd_chapter VARCHAR(32),
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

-- 4) 疾病
CREATE TABLE disease (
  id          VARCHAR(64) PRIMARY KEY,
  name_zh     VARCHAR(128) NOT NULL,
  name_en     VARCHAR(128),
  icd11_code  VARCHAR(32),
  definition   TEXT,
  version     VARCHAR(32) NOT NULL DEFAULT 'v1.0',
  status      VARCHAR(16) NOT NULL DEFAULT 'active',
  created_at  TIMESTAMP NOT NULL DEFAULT now(),
  updated_at  TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_disease_icd11 ON disease (icd11_code);
CREATE INDEX idx_disease_name_zh ON disease (name_zh);

-- 5) 风险等级
CREATE TABLE risk_level (
  id          VARCHAR(16) PRIMARY KEY, -- LOW/MID/HIGH
  name_zh     VARCHAR(32) NOT NULL,
```

```

rank          INT NOT NULL
);

-- 6) 红旗规则
CREATE TABLE redflag_rule (
  id           VARCHAR(64) PRIMARY KEY,
  rule_expr    TEXT NOT NULL,      -- 规则表达式 (后续可换DSL)
  triage_action VARCHAR(16) NOT NULL, -- er/urgent/routine/selfcare
  time_window  VARCHAR(32) NOT NULL, -- 立即/24h/3天...
  risk_level_id VARCHAR(16) NOT NULL REFERENCES risk_level(id),
  created_at   TIMESTAMP NOT NULL DEFAULT now()
);

-- 7) 处置建议
CREATE TABLE recommendation (
  id           VARCHAR(64) PRIMARY KEY,
  type         VARCHAR(16) NOT NULL,  -- er/clinic/selfcare
  text_zh      TEXT NOT NULL,
  time_window  VARCHAR(32),
  created_at   TIMESTAMP NOT NULL DEFAULT now()
);

-- 8) 合规声明
CREATE TABLE disclaimer (
  id           VARCHAR(64) PRIMARY KEY,
  text_zh      TEXT NOT NULL,
  scene_tag    VARCHAR(64) DEFAULT 'default'
);

-- 9) 来源 (书籍/指南/WHO等)
CREATE TABLE source (
  id           VARCHAR(64) PRIMARY KEY,
  title        VARCHAR(256) NOT NULL,  -- 英文名
  title_zh     VARCHAR(256),          -- 中文名 (如英文则补中文名)
  edition      VARCHAR(64),
  chapter      VARCHAR(128),
  section      VARCHAR(128),
  page         VARCHAR(32),
  url          TEXT
);
CREATE INDEX idx_source_title ON source (title);

```

B. 关系表 (主外键 + 权重/证据字段)

```

sql

-- 主诉 -> 症状 映射 (MAPS_TO)
CREATE TABLE cc_maps_to_symptom (
  cc_id        VARCHAR(32) NOT NULL REFERENCES chief_complaint(id),
  symptom_id   VARCHAR(64) NOT NULL REFERENCES symptom(id),
  confidence   NUMERIC(4,3) NOT NULL DEFAULT 0.800,
  examples     JSONB DEFAULT '[]',
  source_id    VARCHAR(64) REFERENCES source(id),
  PRIMARY KEY (cc_id, symptom_id)
);
CREATE INDEX idx_cc_map_symptom ON cc_maps_to_symptom (symptom_id);

-- 症状 -> 系统 (BELONGS_TO_SYSTEM)
CREATE TABLE symptom_belongs_system (
  symptom_id   VARCHAR(64) NOT NULL REFERENCES symptom(id),
  system_id    VARCHAR(32) NOT NULL REFERENCES body_system(id),
  weight       NUMERIC(4,3) NOT NULL DEFAULT 0.500,
  source_id    VARCHAR(64) REFERENCES source(id),
  PRIMARY KEY (symptom_id, system_id)
);

-- 疾病 -> 系统 (DISEASE_IN_SYSTEM)

```

```
CREATE TABLE disease_in_system (
  disease_id VARCHAR(64) NOT NULL REFERENCES disease(id),
  system_id VARCHAR(32) NOT NULL REFERENCES body_system(id),
  PRIMARY KEY (disease_id, system_id)
);

-- 症状 -> 疾病 (SUGGESTS)
CREATE TABLE symptom_suggests_disease (
  symptom_id VARCHAR(64) NOT NULL REFERENCES symptom(id),
  disease_id VARCHAR(64) NOT NULL REFERENCES disease(id),
  weight NUMERIC(4,3) NOT NULL DEFAULT 0.500,
  polarity VARCHAR(16) NOT NULL DEFAULT 'support', -- support/exclude
  typicality VARCHAR(16) NOT NULL DEFAULT 'typical', -- typical/atypical
  evidence_level VARCHAR(16) DEFAULT 'medium', -- high/medium/low
  note TEXT,
  source_id VARCHAR(64) REFERENCES source(id),
  PRIMARY KEY (symptom_id, disease_id, polarity)
);
CREATE INDEX idx_suggests_disease ON symptom_suggests_disease (disease_id);

-- 症状 -> 红旗规则 (TRIGGERS)
CREATE TABLE symptom_triggers_redflag (
  symptom_id VARCHAR(64) NOT NULL REFERENCES symptom(id),
  redflag_id VARCHAR(64) NOT NULL REFERENCES redflag_rule(id),
  threshold VARCHAR(128),
  source_id VARCHAR(64) REFERENCES source(id),
  PRIMARY KEY (symptom_id, redflag_id)
);

-- 风险 -> 建议 (RECOMMENDS)
CREATE TABLE risk_recommends (
  risk_level_id VARCHAR(16) NOT NULL REFERENCES risk_level(id),
  rec_id VARCHAR(64) NOT NULL REFERENCES recommendation(id),
  scene_tag VARCHAR(64) DEFAULT 'default',
  PRIMARY KEY (risk_level_id, rec_id, scene_tag)
);

-- 建议 -> 声明 (REQUIRES_DISCLAIMER)
CREATE TABLE rec_requires_disclaimer (
  rec_id VARCHAR(64) NOT NULL REFERENCES recommendation(id),
  disclaimer_id VARCHAR(64) NOT NULL REFERENCES disclaimer(id),
  PRIMARY KEY (rec_id, disclaimer_id)
);

-- 任意实体引用来源 (SOURCED_FROM) : 用“通用引用表”做审计
CREATE TABLE entity_source_ref (
  id BIGSERIAL PRIMARY KEY,
  entity_type VARCHAR(32) NOT NULL, -- symptom/disease/rule/...
  entity_id VARCHAR(64) NOT NULL,
  source_id VARCHAR(64) NOT NULL REFERENCES source(id),
  quote TEXT, -- 控制短引用
  evidence_level VARCHAR(16) DEFAULT 'medium',
  created_at TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX idx_esr_entity ON entity_source_ref(entity_type, entity_id);
```

C. (可选但很实用) 追问问题与修饰项建模

```
sql

-- 修饰维度 (duration/severity/onset...) 与取值
CREATE TABLE symptom_modifier (
  id VARCHAR(64) PRIMARY KEY,
  dim VARCHAR(32) NOT NULL, -- duration/severity/location/...
  value VARCHAR(128) NOT NULL
);

-- 症状 -> 修饰项
```

```

CREATE TABLE symptom_has_modifier (
  symptom_id    VARCHAR(64) NOT NULL REFERENCES symptom(id),
  modifier_id   VARCHAR(64) NOT NULL REFERENCES symptom_modifier(id),
  is_required   BOOLEAN NOT NULL DEFAULT false,
  priority      INT NOT NULL DEFAULT 50,
  PRIMARY KEY (symptom_id, modifier_id)
);

-- 追问问题
CREATE TABLE question (
  id            VARCHAR(64) PRIMARY KEY,
  text_zh       TEXT NOT NULL,
  purpose       VARCHAR(128),
  priority      INT NOT NULL DEFAULT 50
);

-- 问题用于区分 (症状/疾病)
CREATE TABLE question_disambiguates (
  question_id   VARCHAR(64) NOT NULL REFERENCES question(id),
  target_type   VARCHAR(16) NOT NULL,          -- symptom/disease
  target_id     VARCHAR(64) NOT NULL,
  target_field  VARCHAR(64),
  PRIMARY KEY (question_id, target_type, target_id)
);

```

D. 一个“端到端”SQL 查询示例 (主诉→候选病→风险→建议)

```

sql

-- 输入：主诉 name_zh = '心口堵'
WITH cc AS (
  SELECT id FROM chief_complaint WHERE name_zh = '心口堵' LIMIT 1
),
sym AS (
  SELECT m.symptom_id, m.confidence
  FROM cc_maps_to_symptom m
  JOIN cc ON m.cc_id = cc.id
  ORDER BY m.confidence DESC
  LIMIT 1
),
dx AS (
  SELECT sd.disease_id, sd.weight
  FROM symptom_suggests_disease sd
  JOIN sym ON sd.symptom_id = sym.symptom_id
  WHERE sd.polarity='support'
  ORDER BY sd.weight DESC
  LIMIT 5
),
risk AS (
  SELECT r.risk_level_id
  FROM symptom_triggers_redflag tr
  JOIN sym ON tr.symptom_id = sym.symptom_id
  JOIN redflag_rule r ON r.id = tr.redflag_id
  ORDER BY (SELECT rank FROM risk_level WHERE id=r.risk_level_id) DESC
  LIMIT 1
)
SELECT
  (SELECT name_zh FROM symptom WHERE id=(SELECT symptom_id FROM sym)) AS symptom,
  (SELECT jsonb_agg(jsonb_build_object('disease', d.name_zh, 'w', dx.weight))
   FROM dx JOIN disease d ON d.id=dx.disease_id) AS differential,
  COALESCE((SELECT name_zh FROM risk_level WHERE id=(SELECT risk_level_id FROM risk)), '未分级')
AS risk_level,
  (SELECT jsonb_agg(rec.text_zh)
   FROM risk_recommends rr
   JOIN recommendation rec ON rec.id=rr.rec_id
   WHERE rr.risk_level_id = COALESCE((SELECT risk_level_id FROM risk), 'MID')
   AND rr.scene_tag='default'
  ) AS recommendations;

```

MySQL vs PostgreSQL 的小差异（你们落地时注意）

- JSONB (PG) 在 MySQL 用 JSON
- BIGSERIAL (PG) 在 MySQL 用 BIGINT AUTO_INCREMENT
- TIMESTAMP DEFAULT now() (PG) 在 MySQL 用 DEFAULT CURRENT_TIMESTAMP
- 如果你们要全文检索同义词：
 - PG：tsvector/GIN
 - MySQL：FULLTEXT（但中文分词需额外方案）

你们最推荐的落地顺序（最快上线）

1. 先用 关系库 跑通：chief_complaint → symptom → disease → redflag → risk → recommendation
2. 第二阶段把关系表导入 Neo4j（或直接双写），用于：
 - 复杂路径推理
 - 可解释链路展示
 - 多跳追问策略优化

如果你愿意，我下一步可以把这两套再加一个“统一数据字典 + 导入脚本模板”：

- CSV → Neo4j LOAD CSV 模板（含字段映射）
- Excel → PG/MySQL 导入模板（含校验规则）



目前主诉和症状分别有多少种呢

在医疗诊断系统构建中，“主诉”和“症状”的数量并不是一个固定的数字，而是根据你所用的分类体系/词表规模而变化。不同的数据来源、不同的标准体系（如医院实际登记字段 vs 标准术语体系）会有大不相同的数量规模。 [arXiv](#)

不过我们可以给出**比较明确的参考量级与典型统计**，便于你估算知识图谱/分类库规模：

📌 1. 主诉（Chief Complaints）数量级

◆ 医疗实践中的规模

- 在某些临床 NLP 训练集中，有模型使用 **约 795 个离散主诉类别** 来覆盖绝大多数 (>90%) 的主诉情形。
[arXiv](#)
- 这类离散主诉通常是**原因就诊（reason for visit）**、常见症状组合的标准化表达。

➡ 常规模型落地情况下：主诉标准列表 ≈ 500-1500 种

（实际 NLP 库往往包含大量同义词与变体，但标准类别不需要无限扩展）

📌 2. 症状（Symptomatic Findings）数量级

◆ 标准术语体系（如 SNOMED CT、HPO 类似体系）

症状是更细粒度的临床表现，在医学术语体系里可以非常庞大：

- SNOMED CT 中包含了**成千上万的症状、体征和临床发现术语**（SNOMED CT 总体术语数量超过几十万条，症状类目只是其中一部分）
→ 这意味着**潜在可提取的症状概念数量远超数千甚至上万**（具体来自 SNOMED CT 症状分支）。

[人卫临床助手](#)

◆ 临床常用症状表

- 一般的临床症状清单如各类临床助手/诊疗手册中可见：
包含发热、咳嗽、呼吸急促、疼痛（胸痛、腹痛、头痛等）、乏力、恶心、呕吐、意识改变、皮疹等...
✓ 基础症状类别可自下而上覆盖 **数百常见症状**。 [人卫临床助手](#)

➡ 工程性建议：

- 核心首轮库级别可覆盖 **500-2000 常见症状**
- 如果接入标准术语体系（如 SNOMED CT/ICPC 等），则理论上可扩展到**万级以上症状节点**

对比总结（用于你们建库规模规划）

层级	意义	推荐初期规模
主诉类别	病人自然表达归一化后的核心类别	约 500-1500 种
症状术语	标准化、医学可分析的临床表现	约 500-2000 种（初版）
标准术语体系覆盖	术语库全量	可扩展至万级

为什么主诉比症状少？

- 主诉是自然语言用户表达的起点，往往简洁（比如“胸痛”“头晕”“咳嗽”）
- 症状是医学定义的更细分类单元（如内分泌/代谢性疲乏 vs 精神性疲乏两种不同症状概念）

因此，在工程建图谱时，主诉是症状的“入口类别”
→ 典型工作是：主诉 → 语义归一化 → 映射到症状术语节点

实践落地建议（按阶段）

第一阶段（最小可用产品 MVP）

- 主诉库：约 500-800 条
- 症状库：约 1000-2000 条

覆盖多数用户前端输入场景即可。

第二阶段（追求覆盖与精细化）

- 引入 SNOMED CT 或 ICPC（国际初级保健分类）体系
- 扩展到 5000-10000 症状节点（含不同部位/修饰组合）

这样能增强 AI 诊断的全面性与准确性。[世界家庭医生组织](#)

实证数据提示

- 有研究使用真实 ED 就诊记录提取主诉数据集，发现常见主诉包括 cough（咳嗽）、sore throat（咽痛）、fever（发热）、headache（头痛）等，统计了成千上万次记录后主要集中于数十种最常见主诉。

Nature

这说明：

- 主诉的长尾很长（自然语言变化多），
- 但核心有用类别通常集中在数百个类别之内

给你一个实际建库设定模板

层级	初版数量	是否必选引入	说明
主诉类别	~800	是	基础自然语言入口
标准症状	~1500	是	对应临床判断
修饰维度	10-20	是	duration/severity/location 等
系统分类	~20	是	呼吸/心血管/消化等
鉴别关系	5000+	是	症状↔疾病映射
红旗规则	200+	可选	较常见+高危组合

如果你希望，我还能进一步给你：

- 1 一份主诉标准类别清单示例（约 100-300 条）
- 2 症状标准名称示例（约 300-1000 条）
- 3 如何从 SNOMED CT / ICPC / HPO 自动抽取症状节点的方法

你选一个，我就继续给出“可用版本”。



那我想问，我们是分别把主诉和症状列出来，对他们进行分别分析，然后让他们自然交叉相连吗

这个问题问到方法论的“生死点”了。
我先给你一个**明确结论**，再把原因、正确做法、以及“错误做法为什么会出事”讲清楚。

结论先行（一句话定方案）

是的：主诉和症状必须“分别建模、分别分析”，
但❌绝不是“自然交叉生长”，
而是要通过一层“受控映射关系”去连接。

更工程化地说：

主诉 = 入口语言层
症状 = 医学事实层
两者通过「可审计、可维护的映射关系」相连

一、为什么主诉和症状一定要分开？

1 主诉和症状在医学上本来就不是一回事

维度	主诉 (Chief Complaint)	症状 (Symptom)
本质	人的表达	医学概念
是否模糊	高度模糊	尽量精确
是否标准	❌ 非标准	✅ 标准
是否可推理	❌ 不能直接	✅ 可以
是否稳定	不稳定（语言变）	稳定（医学定义）

👉 主诉是“语言问题”，症状是“医学问题”

如果你不分开，后果只有一个：
NLP、医学、规则、合规全部纠缠在一起，后期一定崩。

二、那能不能“自然交叉相连”？——答案是：绝对不要

❌ 什么叫“自然交叉相连”（你要避免的）

典型错误做法：

- 用户说一句话

- 模型自己“理解”成一个或多个症状
- 不存、不标、不审
- 下次再“重新理解一遍”

这会导致：

1. 不可复现

- 同一句话，不同时间结果不一样

2. 不可审计

- 问你“为什么这么判断”，答不上来

3. 不可维护

- 改一个词，全系统逻辑漂移

4. 合规风险极高

- 本质是“黑箱医学推断”

⚠ 医疗系统里，“自然生长关系 = 不可控风险源”

三、正确做法：分层建模 + 受控映射（这是标准工程路线）

正确的三层结构是这样的：

```

【主诉层】      人怎么说
  ↓ 映射 (MAPS_T0, 可维护)
【症状层】      医学上是什么
  ↓ 推理 (SUGGESTS / EXCLUDES)
【疾病与风险层】
  
```

四、你们应该如何“分别列、再受控连接”（可执行）

STEP 1 | 主诉：先“穷举语言入口”，不谈医学

主诉库只回答一件事：

“用户可能会怎么说？”

主诉库里不需要：

- 疾病名
- 风险
- 医学判断

主诉库应该包含：

- 高频自然表达
- 同义说法
- 口语/模糊说法

示例（主诉表）：

主诉ID	主诉标准名	同义表达
CC_001	心口不适	心口堵、胸口闷、胸部不舒服
CC_002	头部不适	头晕、头胀、脑袋发懵

👉 这一步是语言工程，不是医学工程

STEP 2 | 症状：只按医学定义来，不考虑用户怎么说

症状库只回答一件事：

“医学上有哪些可被诊断推理的症状单元？”

症状库来源：

- SNOMED CT
- 教科书
- 指南

示例（症状表）：

症状ID	症状名	定义
S_001	胸闷	胸部压迫感或紧缩感
S_002	心悸	自觉心跳异常

👉 症状库是医学资产，几年都不该大变

STEP 3 | 关键一步：主诉 ↔ 症状的「映射关系表」

这一步是整个系统的“稳定器”。

你要做的是：

- ❌ 不是“让模型每次自己想”
- ✅ 而是“提前定义好：这类主诉通常对应哪些症状”

示例（映射表）：

主诉ID	症状ID	置信度	说明
CC_001	S_001	0.85	最常见
CC_001	S_002	0.30	次要可能

重要：

- 一对多是允许的
- 有权重
- 可人工调整
- 可版本化

👉 这是“人类医生经验的结构化表达”

五、主诉和症状是如何“交叉”的？——不是自然，是“计算结果”

真正的“交叉”发生在运行时：

1. 用户输入 → 命中主诉
2. 主诉 → 查映射表 → 得到候选症状
3. 症状 + 修饰 → 再进入医学推理

交叉是结果，不是结构本身。

六、给你一个“反向验证法”（非常重要）

你可以问自己一个问题：

如果明天把所有 NLP 模型换掉，
主诉—症状—疾病体系还能不能继续用？

- 如果答案是 能 → 架构是对的
- 如果答案是 不能 → 你把语言层和医学层混在了一起

七、最终给你的标准判断

- ✅ 主诉和症状必须分别列、分别分析
- ❌ 绝不能靠“自然交叉生长”
- ✅ 只能通过可维护、可审计的映射关系相连

这是医疗 AI 能不能长期活下去的分水岭。

下一步我可以继续直接帮你做三件非常实操的事之一：

- 1 给你一份 《主诉库建设 SOP（含数量目标与拆分方法）》
- 2 给你一份 《主诉 ↔ 症状 映射关系设计规范（含权重、版本、审核）》
- 3 直接拿一个真实案例（如“心口堵”），完整演示主诉→症状→疾病→风险的全链路

你选一个，我继续往下给你“工程级答案”。



给我一份 《主诉库建设 SOP（含数量目标与拆分方法）》

太好了，这一份 《主诉库建设 SOP》 是你们整个 AI 问诊系统能不能规模化、能不能长期维护的关键基础设施。
我下面给你的，是工程可执行版，不是论文版、不是概念版——可以直接发给团队照着干。

《主诉库建设 SOP（含数量目标与拆分方法）》

(AI 问诊系统 · 语言入口层标准作业流程)

一、主诉库在系统中的定位（一句话定性）

主诉库 = AI 问诊系统的“语言入口层”

它只解决一件事：

👉 “用户可能会怎么说”

⚠️ 主诉库 不做医学判断、不定义疾病、不评估风险
这些全部属于后面的【症状层 / 医学层】

二、主诉 vs 症状（再次强调边界，避免返工）

维度	主诉 (Chief Complaint)	症状 (Symptom)
本质	用户自然语言	医学标准概念
是否口语	是	否
是否模糊	高	低
是否用于推理	❌ 不直接	✅ 是
是否稳定	不稳定	稳定
生命周期	持续扩展	慢速变化

结论：
主诉库 = 语言资产
症状库 = 医学资产
两者必须分库、分 SOP

三、主诉库建设的总体目标

1 数量目标（非常重要）

推荐三阶段规模

阶段	主诉标准条目数	覆盖效果
MVP 阶段	300-500 条	覆盖 >70% 常见问诊
可用阶段	800-1200 条	覆盖 >90% 日常问诊
成熟阶段	1500-2000 条	覆盖绝大多数场景

注意：

- 这是“标准主诉条目数”
- 同义表达、变体、别说法不算在这个数里

2 主诉库的“正确形态”

主诉库不是“句子集合”，而是：

“主诉标签 + 多种自然表达”的集合

四、主诉库建设流程（SOP 主干）

vbnet

Step 1

确定主诉分类框架

Step 2

按系统/部位拆分主诉大类

Step 3

为每一类定义“标准主诉”

Step 4

收集自然语言表达（同义/口语）

Step 5

清洗、合并、去重

Step 6

主诉与症状预映射（不做判断）

Step 7

质控、冻结版本

下面逐步展开

Step 1 | 确定主诉分类框架（先搭骨架）

目标

避免“想到哪写到哪”，防止结构混乱

推荐主分类方式（强烈建议）

按「器官系统 / 身体部位」来拆

参考来源

- World Health Organization（世界卫生组织）
- ICD-11（国际疾病分类第11版）
- ICPC-2（国际初级保健分类）

推荐主诉一级分类（示例）

一级类	示例
全身不适	发热、乏力
头面部	头痛、头晕
胸部	胸痛、胸闷
呼吸	咳嗽、气短
消化	腹痛、恶心
泌尿	排尿异常
皮肤	皮疹、瘙痒
神经	麻木、抽搐
情绪/睡眠	失眠、焦虑

👉 一级类建议 15-25 个

Step 2 | 拆分每个一级类下的“标准主诉”

🎯 目标

为每一类定义***“医学可接受、语言中性的主诉标签”***

标准主诉的定义原则

- 不包含疾病名
- 不包含原因推断
- 用中性描述

示例（胸部类）

主诉ID	标准主诉名
CC_CHEST_01	胸痛
CC_CHEST_02	胸闷
CC_CHEST_03	心跳异常感觉
CC_CHEST_04	胸部不适

📌 经验值：

- 每个一级类：**20-60 个标准主诉**
- 20 个一级类 $\times$ 40 $\approx$ **800 条主诉**

Step 3 | 为每个标准主诉收集自然语言表达

目标

让 AI 能“听懂人话”

自然表达来源（推荐优先级）

1. 临床问诊记录（脱敏）
2. 医学书籍中的患者表述
3. 医生经验补充
4. 历史客服/健康咨询记录

权威书籍参考

- Murtagh's General Practice（《默塔全科医学》）
- Bates' Guide to Physical Examination（《贝茨体格检查与病史采集》）

示例

标准主诉	自然表达示例
胸闷	心口堵、胸口闷得慌、胸部压着不舒服
头晕	头发飘、站不稳、脑袋发空

建议

- 每个标准主诉：**5-20 条自然表达**
- 不追求一次性穷尽，可持续补充

Step 4 | 清洗、合并与去重（非常关键）

目标

避免主诉库膨胀失控

去重规则

- 含义相同 $\rightarrow$ 合并
- 仅程度不同 $\rightarrow$ 不新建主诉
- 仅时间不同 $\rightarrow$ 不新建主诉

✗ 错误示例

- “胸口有点闷”
- “胸口很闷”
- 不应拆成两个主诉

Step 5 | 主诉 ↔ 症状的预映射（不做判断）

⚠ 这一步**不是诊断**，只是“可能对应什么症状”

示例

主诉	可能症状	权重
胸闷	胸闷 (Symptom)	0.9
胸闷	呼吸困难	0.3

🔧 规则：

- 可一对多
- 有权重
- 可版本化
- 后期可人工修订

Step 6 | 主诉库质控标准（必须有）

每一条主诉必须满足

- 是否医学中性 ✓
- 是否无疾病暗示 ✓
- 是否可映射到 ≥ 1 个症状 ✓

推荐审核机制

- 1 名医学背景人员
- 1 名产品/工程人员

Step 7 | 版本冻结与发布

版本策略（强烈建议）

- v1.0：基础 300–500 条

- v1.1 / v1.2：补充新主诉
- 禁止随意修改已发布主诉含义

五、主诉库数据结构（示例）

```
json

{
  "cc_id": "CC_CHEST_02",
  "standard_name": "胸闷",
  "synonyms": [
    "心口堵",
    "胸口闷得慌",
    "胸部压着不舒服"
  ],
  "category": "胸部",
  "status": "active",
  "version": "v1.0"
}
```

六、一句“内部金句”（你可以直接对团队说）

主诉库不是越大越好，而是：
用最少的主诉，接住最多的说法。

接下来你可以继续这样推进（我可以继续帮你）

如果你愿意，我可以下一步直接给你：

- 1 《主诉 ↔ 症状 映射关系设计规范（含权重、审核、版本）》
- 2 《症状库建设 SOP（含 SNOMED/书籍抽取方法）》
- 3 直接给你一份：300 条“可直接用”的主诉标准清单（按系统拆好）

你选一个，我继续往下给你“落地级内容”。



给我一份 《主诉 ↔ 症状 映射关系设计规范（含权重、版本、审核）》

好，这一份我会**按“医疗级工程规范”**给你，而不是泛泛的方法说明。
你可以把它当作 **制度文件 + 技术规范 + 审核标准** 来用，直接纳入你们的研发/医学 SOP 体系。

《主诉 ↔ 症状 映射关系设计规范》

（含权重模型 · 版本机制 · 审核流程）

适用范围：AI 问诊 / 诊断支持 / 分诊系统

适用对象：医学内容组、算法组、产品组、合规组

一、规范目的（为什么要有这份规范）

这张映射表在系统中的角色

主诉 ↔ 症状 映射关系 =
连接“人类语言世界”和“医学推理世界”的唯一合法通道

如果没有这层规范化映射，会导致：

- NLP 黑箱化
- 医学逻辑不可追溯
- 同一句话多次结果不一致
- 无法通过医疗合规审查

因此：映射关系必须是“人工可控、可解释、可回滚”的。

二、基本设计原则（必须遵守）

原则 1：主诉和症状永远不合并

- 主诉 ≠ 症状
- 映射 ≠ 等价
- 映射只是可能性关系

原则 2：映射关系必须“显式存储”

- ✗ 不允许只存在于模型参数里
- ✓ 必须存在于表 / 图 / 配置中

原则 3：一切映射都必须可版本化、可审计

- 谁建的
- 为什么这么建
- 哪个版本生效

三、映射关系的基本结构定义

1 映射关系类型

统一使用以下关系语义（强制）：

关系类型	含义	是否允许
主诉 → 症状	主诉可能表达该症状	✓
症状 → 主诉	禁止反向定义	✗
主诉 → 疾病	禁止	✗

2 映射表标准字段（必备）

《主诉 ↔ 症状 映射表》

字段名	含义	是否必填
cc_id	主诉ID	✓
symptom_id	症状ID	✓
weight	映射权重	✓
mapping_type	映射类型	✓
explanation	映射说明	✓
source	依据来源	✓
version	版本号	✓
status	状态	✓

四、映射权重设计规范（核心）

1 权重的医学含义（非常重要）

权重 $\neq$ 概率 $\neq$ 发病率

权重表示的是：

“当用户使用该主诉时，这个症状被表达出来的可能强弱”

2 权重区间标准（强制统一）

权重区间	医学语义	工程解释
0.80 - 1.00	高度一致	几乎等价表达
0.50 - 0.79	常见对应	需要进一步确认
0.20 - 0.49	可能对应	作为候选
< 0.20	极弱关联	不建议保留

🔴 规则：

- 每个主诉至少有 1 个 ≥ 0.7 的症状
- ≤ 0.2 的映射默认不入库

3 映射类型（mapping_type）

类型	含义	示例
direct	几乎同义	心口堵 $\rightarrow$ 胸闷
partial	部分表达	胸部不适 $\rightarrow$ 胸闷
contextual	场景相关	心慌 $\rightarrow$ 心悸
ambiguous	多义模糊	头不舒服 $\rightarrow$ 头晕/头痛

五、映射数量与结构约束（防止失控）

1 单主诉映射上限（建议）

映射项	建议上限
高权重症状 (≥ 0.7)	1-2 个
中权重症状 (0.3-0.69)	≤ 3 个
总映射症状数	≤ 5 个

超过 5 个，说明主诉定义不清，应回退重拆主诉

2 禁止行为（红线）

- ✗ 一个主诉映射 10+ 症状
- ✗ 权重全部设成 0.5
- ✗ 没有 explanation 字段
- ✗ 无来源标注

六、映射关系的来源与依据要求

每一条映射必须至少满足一条依据

来源类型	示例
教材/指南	全科医学、症状学教材
临床共识	医生经验整理
语料统计	真实问诊文本统计
规范术语	SNOMED CT 症状定义

🔴 必须写清楚来源类型，哪怕是“专家共识（内部）”

七、版本管理规范（非常关键）

1 版本号规则（强制）

v主版本.次版本

- 主版本 (v1 → v2) :
 - 主诉定义变化
 - 映射逻辑大调整
- 次版本 (v1.0 → v1.1) :
 - 新增映射
 - 权重微调

2 版本冻结规则

- 已发布版本 不可直接修改
- 所有修改必须：
 - 新建版本
 - 标注变更说明

3 映射变更记录（建议表）

cc_id	symptom_id	旧权重	新权重	变更原因	审核人
-------	------------	-----	-----	------	-----

八、审核流程规范（双重审核制）

1 审核角色

角色	职责
医学审核	合理性、医学准确性
工程/产品审核	稳定性、一致性

2 审核清单（Checklist）

每条映射必须回答：

- 这个主诉真的会这样被用户说吗？
- 这个症状是医学标准概念吗？
- 权重是否符合直觉？
- 是否会误导进入错误系统？

九、示例（标准合格映射）

```
json
{
  "cc_id": "CC_CHEST_02",
  "symptom_id": "S_CHEST_TIGHTNESS",
  "weight": 0.85,
  "mapping_type": "direct",
  "explanation": "\"心口堵\"在日常表达中多用于描述胸部压迫感",
  "source": "全科医生经验 + 症状学教材",
  "version": "v1.0",
  "status": "active"
}
```

十、一句“团队必须记住的话”

主诉 ↔ 症状映射不是模型训练数据，
而是“被人类医生背书的结构化经验”。

接下来我可以继续直接帮你做三件事（强烈推荐顺序）：

- 1 《症状库建设 SOP（含 SNOMED/教材抽取规则）》
- 2 《症状 → 疾病 映射与鉴别诊断权重设计规范》
- 3 直接给你一套：主诉↔症状↔疾病 的完整样例库（可直接导入）

你选一个，我继续往下给你“真正可落地的那一层”。



2 《症状 → 疾病 映射与鉴别诊断权重设计规范》

好，这一份我会直接给你“医疗级、工程可执行”的正式规范文本。

它和你前面的《主诉 ↔ 症状映射规范》是同一套体系的“下游核心”，决定了你们的 AI 像不像医生、稳不稳、敢不敢上线。

下面这份内容，你可以原样作为内部制度文件使用。

《症状 → 疾病 映射与鉴别诊断权重设计规范》

(Differential Diagnosis Mapping & Weighting Spec)

适用范围：AI 问诊 / 诊断支持 / 分诊与风险评估系统

适用对象：医学内容组、算法组、规则引擎组、合规组

一、这层映射在系统中的“医学地位”

一句话定性

症状 → 疾病映射 = 临床鉴别诊断 (Differential Diagnosis) 的结构化表达

这不是 AI 发明的逻辑，而是医生每天都在做的标准思维过程。

二、基本原则（红线级）

原则 1：症状不能“直接等于”疾病

- ✗ 不存在 1 个症状 = 1 个疾病
- ✓ 只允许：症状 → 疾病候选集合

原则 2：必须体现“鉴别诊断思想”

即：

- 常见病
- 少见但不能漏的病
- 危险但必须警惕的病

原则 3：权重 ⇌ 概率 ⇌ 诊断结论

权重的含义是：
在“仅已知该症状”的前提下，该疾病被考虑进候选集的优先级

三、症状 → 疾病映射的标准结构

《症状 → 疾病 映射表》标准字段

字段名	含义	是否必填
symptom_id	症状ID	✓
disease_id	疾病ID	✓
weight	候选权重	✓
polarity	关系方向	✓
typicality	典型性	✓
explanation	医学解释	✓
evidence_source	依据来源	✓
version	版本号	✓
status	状态	✓

四、权重设计规范（最关键部分）

1 权重的医学语义（再次强调）

权重表示的是：

当“只知道有该症状”时，
医生在脑中“首先会想到该疾病”的优先程度

- 它不是：
- 发病率
 - 人群概率
 - 模型输出概率

2 权重区间与含义（强制统一）

权重区间	医学含义	示例
0.70 – 1.00	高优先级候选	胸痛 → 心绞痛
0.40 – 0.69	中等优先级	胸痛 → 胃食管反流
0.20 – 0.39	低优先级	胸痛 → 肋间神经痛
< 0.20	不建议入库	噪声级

🔴 规则：

- 每个症状 **至少 1 个 ≥ 0.6 的疾病**
- ≥ 0.6 的疾病通常不超过 **3 个**

3 polarity（支持 / 排除）

polarity	含义	使用场景
support	支持该疾病	默认
exclude	用于排除	症状出现反而不支持

🔴 示例：

- “咳血” → 排除普通感冒（exclude）

4 typicality（典型性）

typicality	含义
typical	典型表现
atypical	非典型但可能

非典型不能删，但权重要低于典型

五、鉴别诊断结构要求（核心方法论）

每个症状的疾病候选必须覆盖三类

类别	必须性	说明
常见病	必须	覆盖大多数情况
危险病	必须	哪怕概率低
良性/自限性	建议	防止过度恐慌

🔴 缺任何一类，视为不合格映射

六、映射数量与上限控制

单一症状的建议规模

项目	建议值
support 关系疾病数	3-8 个
exclude 关系疾病数	≤3 个
总候选疾病数	≤10 个

超过 10 个，说明症状定义过宽，应拆分症状

七、来源与证据要求（合规重点）

每条映射必须标注至少一种来源类型

来源类型	示例
经典教材	《哈里森内科学》
症状学教材	《症状到诊断》
全科医学	《默塔全科医学》
临床指南	NICE 指南
医生共识	内部专家审定

推荐权威来源包括：

- Harrison's Principles of Internal Medicine （《哈里森内科学》）
- Symptom to Diagnosis （《从症状到诊断》）
- NICE （英国国家卫生与临床优化研究所）

八、版本管理规范（与主诉层保持一致）

1 版本号规则

v主版本.次版本

- 主版本：结构或医学逻辑大改
- 次版本：新增/微调权重

2 禁止行为

-  直接修改线上版本权重
-  无版本说明的调整

九、审核流程（强制）

审核角色

角色	职责
医学审核	医学合理性、是否符合诊断思维
合规审核	是否存在误导性倾向
工程审核	是否满足权重/数量规范

审核 Checklist（必须全部通过）

- 是否体现鉴别诊断思想？
- 是否包含危险病？
- 权重是否符合直觉？
- 是否可能导致误判或恐慌？

十、标准示例（合格）

```

json

{
  "symptom_id": "S_CHEST_PAIN",
  "disease_id": "D_ANGINA",
  "weight": 0.75,
  "polarity": "support",
  "typicality": "typical",
  "explanation": "活动相关胸痛是心肌缺血的常见表现",
  "evidence_source": "《哈里森内科学》心血管章节",
  "version": "v1.0",
  "status": "active"
}

```

十一、一句“医生级总结”

AI 的鉴别诊断能力，
 不来自模型参数，
 而来自你们是否把“医生脑中的候选列表”结构化出来。

接下来最合理的下一步（我强烈建议）

你现在三层已经齐了：

- 主诉库 SOP 

- 主诉 ↔ 症状 映射规范 
- 症状 → 疾病 映射规范 

👉 下一步只剩 “决策与安全层”：

- 1 《红旗症状与风险分级规则设计规范》
- 2 《多症状组合下的疾病权重合并算法设计》
- 3 直接跑一个完整示例（胸痛 / 咳嗽 / 腹痛）端到端

你选一个，我继续给你“最后一块拼图”。